

depth is measured from root and height is measured from leaf.

Tree

Full binary tree = Children 0/2.

Complete binary tree

Class Node {

int data;

Node left;

Node right;

3

class BinaryTree {

public Node ^{new} createNode(int k) {

Node a = new Node();

a.data = k;

a.left = null;

a.right = null;

return a;

3

P S V m(String[] args) {

Node root = a.createNewNode(2);

root.left = 7

root.right = 5

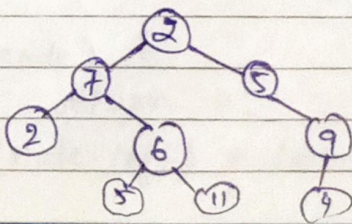
root.left.right = 8

root.left.right.left = 2

root.left.right.right = 11

root.right.right = 9

root.right.right.left = 4



// Traversal.

```
Public void inorder(Node node) {  
    if (node == null)  
        return;  
  
    inorder(node.left);  
    System.out.print (node.data + " ");  
    inorder (node.right);  
}
```

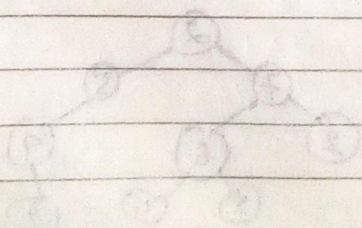
3

```
Public void preorder (Node node) {  
    if (node == null)  
        return;  
  
    System.out.print (node.data + " ");  
    Preorder (node.left);  
    Preorder (node.right);  
}
```

3

```
Public void postorder (Node node) {  
    if (node == null)  
        return;  
  
    postorder (node.left);  
    postorder (node.right);  
    S.O. print (node.data + " ");  
}
```

3



// Sum of all Nodes.

```
Public int getSumOfNode(Node node) {
```

```
    if (node == null)
        return 0;
```

```
    return node.data + getSumOfNode(node.left) +
           getSumOfNode(node.right);
```

// Get difference of values at Even & odd level

```
Public int getDifferenceEOValues(Node node) {
```

```
    if (node == null)
        return 0;
```

```
    return node.data - getDifferenceEOValues(node.left)
           - getDifferenceEOValues(node.right);
```

3

// Get Numbers of Nodes in a Binary tree

```
Public int count(Node node) {
```

```
    if (node == null) return 0;
```

```
    return 1 + count(node.left) + count(node.right);
```

3

11 Get numbers of leaf Nodes in Binary tree

```
public int countleafnode (Node node) {
```

```
    if (node == null)
        return 0;
```

```
    if (node.left == null && node.right == null)
        return 1;
```

```
    return countleafnode (node.left) + countleafnode (node.right);
```

3

11 Get Height of a Binary tree

```
public int getHeightOfBinaryTree (Node node) {
```

```
    if (node == null)
        return -1;
```

```
    return max (getHeightOfTree (node.left),
                getHeightOfTree (node.right)) + 1;
```

3

अगर root के बच्चे कुछ नहीं होंगे तो height 0 होगा.

level starts with
1.
height starts with 0.

// print elements at given level

```
public void printAtGivenLevel(Node node, int level) {
```

```
    if (node == null)
        return;
```

```
    if (level == 1) {
        s.o.p(node.data + " ");
        return;
```

3

```
        printAtGivenLevel(node.left, level - 1);
        printAtGivenLevel(node.right, level - 1);
```

3

// print elements in level order (Recursion)

```
public void levelOrderTraversalUsingRecursion(Node node) {
    if (node == null)
        return;
```

```
    int height = getHeightOfBinary(node);
    for (int i = 0; i <= height; i++) {
        printAtGivenLevel(node, i + 1);
```

3

3

// Level Order traversal w/o Recursion (using Queue)

```
public void levelOrderTraversalUsingQueue(Node node) {
```

```
    if (node == null)
```

```
        return;
```

```
    Queue<Node> q = new LinkedList<Node>();
```

```
    q.add(node);
```

```
    while (q.size() > 0) {
```

```
        Node top = q.remove();
```

```
        System.out.print(top.data + " ");
```

```
        if (top.left != null)
```

```
            q.add(top.left);
```

```
        if (top.right != null)
```

```
            q.add(top.right);
```

3

// Print Reverse level order Traversal (Using Recursion)

```
public void RLOTUR(Node node) {
```

```
    if (node == null)
```

```
        return;
```

```
    int height = getHeightOfTree(node);
```

```
    for (int i = height; i >= 0; i--) {
```

```
        PrintAtGivenLevel(node, i+1);
```

```
        System.out.println();
```

3

// Reverse Level Order Traversal w/o using Recursion.

```
Public void rLDTWDR (Node node) {
```

```
    if (node == null)
        return;
```

```
    Queue<Node> q = new LinkedList<Node>();
    q.add(node);
```

```
    Stack<Node> s = new Stack<Node>();
```

```
    while (q.size() > 0) {
```

```
        Node t = q.remove();
```

```
        if (t.right != null)
            q.add(t.right);
```

```
        if (t.left != null)
            q.add(t.left);
```

```
        s.push(t);
```

```
    }
```

```
    while (s.size() > 0) {
```

```
        sO.print (s.pop().data + " ");
```

```
    }
```

// level Order Traversal Line by Line w/o Recursion

```
public void LDTLN(Node node) {
```

```
    if (node == null)
```

```
        return;
```

```
    Queue<Node> q = new LinkedList<Node>();
```

```
    q.add(node);
```

```
    while (true) {
```

```
        int count = q.size();
```

```
        if (count == 0)
```

```
            break;
```

```
        while (count > 0) {
```

```
            Node top = q.remove();
```

```
            S.o.Print(top.data + " ");
```

```
            if (top.left != null)
```

```
                q.add(top.left);
```

```
            if (top.right != null)
```

```
                q.add(top.right);
```

```
            count--;
```

```
        }
```

```
        S.o.println();
```

// Print left view of Binary tree

int maxlevel;

public void leftViewOfTree (Node node, int level) {

if (node == null)

return;

if (level >= maxlevel) {

S.o.Print (node.data + " ");

maxlevel++;

3

leftViewOfTree (node.left, level + 1);

" (node.right, level + 1);

3

// Print Right view of Binary tree.

int maxlevel;

public void rightViewOfTree (Node node, int level) {

if (node == null)

return;

if (level >= maxlevel) {

S.o.Print (node.data + " ");

maxlevel++;

3

rightViewOfTree (node.right, level + 1);

right " " " (node.left, " ");

3

// Print Inorder Traversal w/o Recursion.

```
public void inorderUsingStack(Node node) {
```

```
    if (node == null)
        return;
```

```
    Stack<Node> s = new Stack<Node>();
```

```
    while (node != null) {
```

```
        s.push(node);
```

```
        node = node.left;
```

```
    }
```

```
    while (s.size() > 0) {
```

```
        Node t = s.pop();
```

```
        s.o.print(t.data + " ");
```

```
        if (t.right != null) {
```

```
            Node tmp = t.right;
```

```
            while (tmp != null) {
```

```
                s.push(tmp);
```

```
                tmp = tmp.left;
```

```
            }
```

```
        }
```

```
    }
```

// Print PreOrder Traversal w/o using recursion.

```
public void PreOrderWithoutRecursion (Node node) {  
    if (node == null)  
        return;
```

```
    Stack<Node> s = new Stack<Node>();  
    s.push(node);
```

```
    while (s.size() > 0) {  
        Node t = s.pop();  
        s.o.Print (t.data + " ");
```

```
        if (t.right != null)
```

```
            s.push(t.right);
```

```
        if (t.left != null)
```

```
            s.push(t.left);
```

3

3

// PostOrder Traversal w/o using recursion.

```
public void PostOrderUsingTwoStack (Node node) {  
    if (node == null) return;
```

```
    Stack<Node> s1 = new Stack<Node>();
```

```
    "      s2 = " " " "
```

```
    s1.push(node);
```

```
    while (s1.size() > 0) {
```

```
        Node t = s1.pop();
```

```
        if (t.left != null) s1.push(t.left);
```

```
        right " " right);
```

```
        s2.push(t);
```

3

```
    while (s2.size() > 0) s.o.Print (s2.pop().data() + " ");
```

3

// Convert a Binary tree to its Mirror Tree.

```
public Node mirrorTree (Node node) {  
    if (node == null) return null;  
  
    Node t = node.left;  
    node.left = node.right;  
    node.right = t;  
  
    mirrorTree (node.left);  
    mirrorTree (node.right);  
    return node;  
}
```

3

// Delete the Binary tree

```
public Node deleteTree (Node node) {  
    if (node == null) return null;  
  
    node.left = deleteTree (node.left);  
    node.right = deleteTree (node.right);  
  
    s.o.p ("Deleting node " + node.data);  
    node = null;  
    return node;  
}
```

3

// Check if two Binary trees are identical

```
public boolean identical (Node t1, Node t2) {  
    if (t1 == null & t2 == null) return true;  
    if (t1 == null || t2 == null) return false;  
    return t1.data == t2.data && identical (t1.left, t2.left)  
        && identical (t1.right, t2.right);  
}
```

3

// Get level of a Given Node

```
Public int getLON(Node node, int val, int level) {  
    if (node == null) return 0;
```

```
    int l;
```

```
    if (node.data == val) return level;
```

```
    l = getLON(node.left, val, level+1);
```

```
    if (l != 0) return l;
```

```
    l = getLON getLON(node.right, val, level+1);
```

```
    return l;
```

3

// print top view of Binary tree

```
Public void topview(Node node) {
```

```
    if (node == null) return;
```

```
    TreeMap<Integer, Integer> m = new TreeMap<>();
```

```
    Queue<Node> q = new LinkedList<Node>();
```

```
    Class Node {  
        Node left;  
        Node right;  
        int data;  
        int height;
```

```
        while (!q.isEmpty()) {
```

```
            Node tmp = q.remove();
```

```
            int hd = tmp.height;
```

```
            if (m.get(hd) == null) m.put(hd, tmp.data);
```

```
            if (tmp.left != null) {
```

```
                tmp.left.height = hd - 1;
```

```
                q.add(tmp.left)
```

3

```

if (tmp.right != null) {
    tmp.right.height = hd + 1;
    q.add(tmp.right);
}

```

```

}
s.o.p(m.values());

```

3

// print Bottom view of Binary tree

```

Public void bottomView(Node node) {
    if (node == null) return;
}

```

```

TreeMap<Integer, Integer> m = new TreeMap<>();
Queue<Node> q = new LinkedList<Node>();
q.add(node);

```

```

while (!q.isEmpty()) {
    Node tmp = q.remove();
    int hd = tmp.height;
    m.put(hd, tmp.data);
}

```

```

if (tmp.left != null) {
    tmp.left.height = hd - 1;
    q.add(tmp.left);
}

```

3

```

if (tmp.right != null) {
    tmp.right.height = hd + 1;
    q.add(tmp.right);
}

```

```

} s.o.p(m.values());

```

// Print Boundary Nodes in Binary tree

```
Public void printBoundary(Node node) {  
    if (node != null) {  
        S.O.P (node.data + " ");  
        printBoundaryLeft (node.left);  
        printLeaves (node.left);  
        printLeaves (node.right);  
        printBoundaryRight (node.right);  
    }  
}
```

3

```
Public void printBoundaryLeft (Node node) {  
    if (node == null) return;
```

```
    if (node.left != null) {  
        S.O.P (node.data + " ");  
        printBoundaryLeft (node.left);  
    }  
    else if (node.right != null) {  
        S.O.P (node.data + " ");  
        printBoundaryLeft (node.right);  
    }  
}
```

3

3

```
Public void printLeaves (Node node) {  
    if (node == null) return;
```

```
    printLeaves (node.left);
```

```
    if (node.left == null && node.right == null)  
        S.O.P (node.data + " ");
```

```
    printLeaves (node.right);  
}
```

3

```
Public void printBoundaryRight(Node node) {
```

```
    if (node == null) return;
```

```
    if (node.right != null) {
```

```
        printBoundaryRight(node.right);
```

```
        S.O.P(node.data + " ");
```

```
    }
```

```
    else if (node.left != null) {
```

```
        printBoundaryRight(node.left);
```

```
        S.O.P(node.data + " ");
```

```
    }
```

```
}
```

```
// Print vertical order of Binary tree
```

```
Public void printVerticalOrder(Node node) {
```

```
    if (node == null) return;
```

```
class Node { int hd = 0; ArrayList<Integer>
```

```
Node left; TreeMap<Integer, Integer> m = new TreeMap<>();
```

```
Node right; getVerticalOrder(node, hd, m);
```

```
int data;
```

```
for (Map.Entry<Integer, ArrayList<Integer>> cm:
```

```
    m.entrySet()) {
```

```
        S.O.P(cm.getValue());
```

```
    }
```

```
}
```

```
Public void getVerticalOrder(Node node, int hd,
```

```
TreeMap<Integer, ArrayList<Integer>> m) {
```

```
    If (node == null) return;
```

```

if (m.get(hd) == null) {
    ArrayList<Integer> l = new ArrayList<>();
    l.add(node.data);
    m.put(hd, l);
}

```

```

} else {
    ArrayList<Integer> l = m.get(hd);
    l.add(node.data);
    m.put(hd, l);
}

```

```

getVerticalOrder(node.left, hd-1, m);
getVerticalOrder(node.right, hd+1, m);

```

3

// Get vertical Sum of Binary Tree:

// Get Sum of elements level wise of Binary tree

// Search an element in Binary tree.

```

public boolean recursiveSearch(Node node, int key) {
    if (node == null) return false;

```

```

    if (node.data == key) return true;

```

```

    return recursiveSearch(node.left, key) ||
           recursiveSearch(node.right, key);

```

3

```

public boolean iterativeSearch(Node node, int key) {
    if (node == null) return false;

```

```

    Queue<Node> q = new LinkedList<Node>();
    q.add(node);

```

```

    while (!q.isEmpty()) {
        Node t = q.remove();
        if (t.data == key) return true;
        if (t.left != null) q.add(t.left);
        if (t.right != null) q.add(t.right);
    }
    return false;

```

3

11. Print Binary Tree in Spiral Orders

```

public void printInSpiralForm(Node node) {
    Stack<Node> s1 = new Stack<>();
    Stack<Node> s2 = new Stack<>();
    s1.push(node);

```

```

    while (!s1.isEmpty() || !s2.isEmpty()) {

```

```

        while (!s1.isEmpty()) {

```

```

            Node tmp = s1.pop();

```

```

            s.o.p(tmp.data + " ");

```

```

            if (tmp.left != null)

```

```

                s2.push(tmp.left);

```

```

            if (tmp.right != null)

```

```

                s2.push(tmp.right);

```

3

```

            s.o.p();

```

```
while (!S2.isEmpty()) {
```

```
    Node tmp = S2.pop();
```

```
    S.o.p(tmp.data + " ");
```

```
    if (tmp.right != null) S1.push(tmp.right);
```

```
    if (tmp.left != null) S1.push(tmp.left);
```

```
}
```

```
S.o.p();
```

```
}
```

```
}
```

// Print elements between any two given level of Binary

```
public void printBTL(Node node, int min, int max) {
```

```
    if (node == null) return;
```

```
    Queue<Node> q = new LinkedList<Node>();
```

```
    q.add(node);
```

```
    int l = 1;
```

```
    while (true) {
```

```
        int size = q.size();
```

```
        if (size == 0 || l > max) break;
```

```
        while (size > 0) {
```

```
            Node t = q.remove();
```

```
            if (l >= min && l <= max)
```

```
                S.o.p(t.data + " ");
```

```
            if (t.left != null) q.add(t.left);
```

```
            if (t.right != null) q.add(t.right);
```

```
            size--;
```

```
        }
```

```
        l++;
```

```
}
```

```
}
```

// maximum width of a Binary tree.

```
public int getMaxWidth(Node node) {  
    if (node == null) return 0;
```

```
    Queue<Node> q = new LinkedList<Node>();  
    q.add(node);
```

```
    int maxWidth = 1;
```

```
    while (true) {
```

```
        int size = q.size();
```

```
        if (size == 0) break;
```

```
        if (size > maxWidth) maxWidth = size;
```

```
        while (size > 0) {
```

```
            Node t = q.remove();
```

```
            if (t.left != null) q.add(t.left);
```

```
            if (t.right != null) q.add(t.right);
```

```
            size--;
```

```
        }
```

```
    }
```

```
    return maxWidth;
```

```
}
```

// Check if two trees are mirror to each other.

// Check if two trees are mirror structure to each other.

// Check if two Binary trees have same structure

// Check if Binary tree is foldable tree

// Check if two Binary trees are isomorphic

// Get width of a level in Binary tree.

```

public int getWidthOfLevel(Node node, int level) {
    if (node == null) return 0;
    if (level == 1) return 1;
    return getWidthOfLevel(node.left, level-1) +
           getWidthOfLevel(node.right, level-1);
}

```

3

// Create Double tree of given Binary tree

```

public void doubleTree(Node node)
    if (node == null) return;

    doubleTree(node.left);
    doubleTree(node.right);

```

```

Node newNode = CreateNewNode(node.data);
newNode.left = node.left;
node.left = newNode;

```

3

// Create Binary tree from Inorder & PreOrder

```
int preIndex;
```

```

public Node buildTreeFromInoP(int P[], int i[], int s, int e)
    if (s > end) return null;

```

```

Node node = CreateNewNode(P[preIndex++]);
if (start == end) return node;

```

inorder index

```
int iind = getInorderIndex(node.data, i, s, e);
```

```
node.left = buildTreeFromInDP(p, i, s, iind-1);
node.right = buildTreeFromInDP(p, iind, i+1, e);
return node;
```

3

```
public int getInorderIndex(int val, int i[], int s, int e)
```

```
{
    for (int j = s; j <= e; j++)
        if (i[j] == val)
            return j;
}
```

3

```
return -1;
```

3

// Get Height of Binary Tree in Iterative Manner

// Get Diameters of Binary tree (max distance between any two leaf nodes)

```
int diameter;
```

```
public int getDiameter(Node node) {
    if (node == null) return 0;
    if (node.left == null && node.right == null)
        return 1;
    int lh = getDiameter(node.left);
    int rh = getDiameter(node.right);
```

3

3

diameter = Math.max(diameter, lh + rh + 1);
return Math.max(lh, rh) + 1;

11 Check if given Binary Tree is Height Balanced Tree

```
Public int ifHeightBalancedTree(Node node) {  
    if (node == null) return 0;  
    if (node.left == null && node.right == null) return 1;  
  
    int lh = ifHeightBalancedTree(node.left);  
    int rh = ifHeightBalancedTree(node.right);  
  
    if (lh == -1 || rh == -1 || Math.abs(lh - rh) > 1)  
        return -1;  
  
    return Math.max(lh, rh) + 1;  
}
```

3

11 Check if Root to Leaf Sum matches for Given Sum in Binary tree

```
Public boolean ifRootTLPSM(Node node, int total) {  
    if (node == null) return false;  
  
    if (node.left == null && node.right == null && total == node.data)  
        return true;  
  
    return ifRootTLPSM(node.left, total - node.data) ||  
           ifRootTLPSM(node.right, total - node.data);  
}
```

3

- 11 Find maximum or minimum value in Binary tree.
- 11 Print root to leaf every Path in Binary tree.
- 11 Get Max Sum from Root to leaf in Binary tree.
- 11 Get Sum of all numbers formed from Root to leaf

- // Find min Height of Binary tree
- // Get Sum of all leaf Nodes in Binary tree
- // Sum of all left leaf Nodes

// Get minimum distance to leaf from Root

// Check if all leaf Nodes are at Same level

// Get deepest left leaf Node

// Find deepest odd level which contains leaf node

int depthDeepestOddLevelNode;

public void depthDeepestOddLevelNodeSecond(Node node, int level)

if (node == null) return;

if (node.left == null && node.right == null && (level % 2 != 0) && level > depthDee)

depthDee = level;

depthDeepestOddLevelNodeSecond(node.left, level+1);

node.right, " ");

3

// Remove all Nodes which lies on Path with less than k length from Root to leaf

public Node removeNodesLessThanKLength(Node node, int k, int level) {

if (node == null) return;

node.left = removeNodesLessThanKLength(node.left, k, level+1);

node.right =

removeNodesLessThanKLength(node.right, k, level+1);

if (node.left == null && node.right == null && index == seq.length - 1) return true; else if (node.left != null) return checkIfSequence(node.left, seq, index + 1); else if (node.right != null) return checkIfSequence(node.right, seq, index + 1); else return false;

11 Remove nodes which don't lie in any path from root to leaf with sum greater than

11 Find if a given sequence is present as a root to leaf path in the given Binary tree.

```
public boolean checkIfSequence (Node node, int[] seq,
                                int index) {
    if (node == null) return false;
    if (index >= seq.length || node.data != seq[index])
        return false;
    return checkIfSequence (node.left, seq, index + 1) ||
           checkIfSequence (node.right, seq, index + 1);
}
```

3

11 Print ancestors of given value in binary tree.

11 Get Parent of a given value in Binary tree.

11 Get Sibling node of a given value in Binary tree.

11 Check if given two nodes are Siblings in BT

```
return (node.left == a && node.right == b) || (n.l == b && (n.r == a));
11 isSiblings (node.left, a, b) || isSiblings (node.right, a, b);
```

⇒ same level or not.

11 Check if two given nodes are cousin to each others

⇓
not sibling (sibling $\nrightarrow$ $\frac{a}{b}$ $\frac{c}{d}$ $\frac{e}{f}$ $\frac{g}{h}$)

- 11 Find distance b/w two nodes in Binary tree.
- 11 Print all nodes at K distance from given node in Binary tree.
- 11 Print all nodes at K distance from leaf node in Binary tree.
- 11 Get closest leaf node distance from a given value in Binary tree.
- 11 Get Right node of given value in Binary tree.
- 11 Connect Nodes at Same Level with next pointers.
- 11 Print all nodes at K distance from given node in Binary tree.

```

Public int PEAKD(Node node, Node targetNode, int k) {
    if (node == null || targetNode == null || k < 0)
        return -1;
    return PEAKDU(node, targetNode, k);
}

```

3

```

Public int PEAKDU(Node node, Node targetNode, int k) {
    if (node == null)
        return -1;
    if (node == targetNode) {
        PNAKDFromParent(node, k);
        return 0;
    }
}

```

3

```
int leftDist = PEAKDU(node.left, targetNode, K);
```

```
if (leftDist != -1) {
```

```
    if (leftDist + 1 == K)
```

```
        S.O.P(node.data + " ");
```

```
    else
```

```
        PNAKDfromParent(node.right, K-2-leftDist);
```

```
    return leftDist + 1;
```

```
3
```

```
int rightDist = PEAKDU(node.right, targetNode, K);
```

```
if (rightDist != -1) {
```

```
    if (rightDist + 1 == K)
```

```
        S.O.P(node.data + " ");
```

```
    else
```

```
        PNAKDfromParent(node.left, K-2-rightDist);
```

```
    return rightDist + 1;
```

```
3
```

```
return -1;
```

```
3
```